

UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL
CUSCO

CARRERA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE
SISTEMAS

GUÍA DE LABORATORIO N.º 04

Montículos de Fibonacci y análisis amortizado

Asignatura:	Algoritmos Avanzados
Unidad:	II. Estructuras de datos avanzadas
Semestre académico:	2026-II
Duración:	120 minutos
Organización:	Trabajo grupal en los equipos del proyecto semes- tral
Integrantes:	Aproximadamente cuatro estudiantes por equipo
Lenguaje:	Python 3
Docente:	Héctor Eduardo Ugarte Rojas

Equipo: _____

Integrantes: _____

Fecha: _____

Índice

1. Objetivo de la práctica	2
2. Trabajo preparatorio obligatorio	2
3. Materiales	3
4. Marco teórico breve	3
4.1. Colas de prioridad y montículos	3
4.2. Consolidación, marcas y cortes	3
4.3. Análisis amortizado mediante potencial	3
4.4. Comparación con <code>heapq</code>	4
5. Especificación mínima de la implementación	4
5.1. Clase de nodo	4
5.2. Clase del montículo	4
6. Invariantes que deben comprobarse	4
7. Distribución sugerida de los 120 minutos	5
8. Ejercicios resueltos	5
8.1. Ejercicio resuelto 1: instrumentar la función potencial	5
8.2. Ejercicio resuelto 2: consolidación programada por grados	6
8.3. Ejercicio resuelto 3: simular <code>decrease-key</code> con <code>heapq</code>	7
9. Ejercicios propuestos	8
9.1. Ejercicio propuesto 1: implementar y validar el montículo de Fibonacci	8
9.2. Ejercicio propuesto 2: comparar con <code>heapq</code> y aplicar en Dijkstra	9
10. Metodología de medición	10
11. Resultados mínimos requeridos	11
12. Preguntas de análisis	11
13. Organización del trabajo grupal	11
14. Producto de entrega	11
15. Tabla de evaluación	12
16. Casos mínimos de prueba	12
17. Referencias	13

1. Objetivo de la práctica

Objetivo

Implementar las operaciones esenciales de un montículo de Fibonacci, verificar sus invariantes estructurales y relacionar el costo real de las operaciones con la función potencial. Posteriormente, comparar experimentalmente la implementación desarrollada con `heapq` de Python, distinguiendo complejidad teórica, costo amortizado y rendimiento práctico.

Al finalizar la práctica, el equipo será capaz de:

1. representar nodos, listas circulares de raíces y relaciones padre-hijo;
2. implementar `insert`, `minimum`, `extract-min`, `decrease-key`, `delete` y `union`;
3. realizar consolidaciones por grado y cortes en cascada;
4. comprobar automáticamente las invariantes del montículo;
5. calcular $t(H)$, $m(H)$ y $\Phi(H) = t(H) + 2m(H)$;
6. registrar costo real y cambio de potencial en una secuencia;
7. comparar un montículo de Fibonacci con un montículo binario basado en `heapq`;
8. interpretar por qué una mejor cota asintótica no garantiza menor tiempo para toda instancia.

2. Trabajo preparatorio obligatorio

Cada integrante deberá revisar:

1. la propiedad de montículo mínimo;
2. la diferencia entre costo real, peor caso y costo amortizado;
3. la función potencial:
$$\Phi(H) = t(H) + 2m(H);$$
4. el procedimiento de consolidación de raíces con el mismo grado;
5. la regla de marcas y cortes en cascada;
6. las operaciones básicas de `heapq`: `heappush`, `heappop` y `heapify`.

Alcance de la práctica

La implementación completa no debe iniciarse desde cero durante los 120 minutos. El equipo deberá llegar con las clases básicas preparadas. En el laboratorio se completarán, probarán y analizarán las operaciones críticas, especialmente `extract-min`, `decrease-key` y la comparación experimental.

3. Materiales

- Computadora con Python 3.11 o posterior, recomendado.
- Editor o IDE, terminal y Git.
- Biblioteca estándar: `heapq`, `itertools`, `random`, `time`, `statistics`, `csv` y `math`.
- Repositorio del equipo con la estructura solicitada.
- Diapositivas del tema 4 y casos de prueba preparados por el equipo.
- Herramienta para generar tablas y gráficos a partir del archivo CSV.

4. Marco teórico breve

4.1. Colas de prioridad y montículos

Una cola de prioridad mínima procesa primero el elemento cuya clave es menor. Un montículo binario, como el utilizado por `heapq`, representa implícitamente un único árbol binario dentro de una lista. Un montículo de Fibonacci mantiene un bosque de árboles de grado variable, una lista circular de raíces y un puntero al mínimo.

Las operaciones más relevantes para esta práctica son:

Operación	Montículo binario	Fibonacci, amortizado
<code>insert</code>	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$
<code>minimum</code>	$\mathcal{O}(1)$	$\mathcal{O}(1)$
<code>extract-min</code>	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
<code>decrease-key</code>	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$
<code>union</code>	no es constante	$\mathcal{O}(1)$

4.2. Consolidación, marcas y cortes

Después de `extract-min`, las raíces con el mismo grado se enlazan hasta conservar como máximo una raíz por grado. Al disminuir una clave, un nodo que queda por debajo de su padre se corta y pasa a la lista de raíces. Un nodo no raíz se marca cuando pierde por primera vez un hijo; si pierde otro hijo mientras está marcado, también se corta y el procedimiento puede continuar hacia arriba.

4.3. Análisis amortizado mediante potencial

El costo amortizado de la operación i se expresa como:

$$\hat{c}_i = c_i + \Phi(H_i) - \Phi(H_{i-1}),$$

donde c_i es el costo real simplificado. Para el montículo de Fibonacci se utiliza:

$$\Phi(H) = t(H) + 2m(H),$$

con $t(H)$ igual a la cantidad de árboles y $m(H)$ a la cantidad de nodos marcados. El potencial no es tiempo de reloj: representa trabajo estructural pendiente y permite distribuir el costo de operaciones ocasionalmente caras dentro de una secuencia.

4.4. Comparación con heapq

heapq ofrece inserción y extracción, pero no una operación directa de disminución de clave sobre un elemento arbitrario. Para compararlo con el montículo de Fibonacci se utilizarán actualizaciones perezosas: la entrada anterior se invalida y se inserta otra con la nueva prioridad. Esta técnica preserva el resultado lógico, pero puede dejar entradas obsoletas en el arreglo interno.

5. Especificación mínima de la implementación

5.1. Clase de nodo

Cada nodo deberá almacenar, como mínimo:

Listing 1: Interfaz mínima del nodo

```
1 class FibonacciNode:
2     def __init__(self, key, value=None):
3         self.key = key
4         self.value = value
5         self.parent = None
6         self.child = None
7         self.left = self
8         self.right = self
9         self.degree = 0
10        self.mark = False
```

5.2. Clase del montículo

La clase deberá mantener:

Listing 2: Interfaz requerida del montículo

```
1 class FibonacciHeap:
2     def __init__(self): ...
3     def is_empty(self): ...
4     def minimum(self): ...
5     def insert(self, key, value=None): ...
6     def union(self, other): ...
7     def extract_min(self): ...
8     def decrease_key(self, node, new_key): ...
9     def delete(self, node): ...
10
11     # Metodos internos
12     def _consolidate(self): ...
13     def _link(self, child, parent): ...
14     def _cut(self, node, parent): ...
15     def _cascading_cut(self, node): ...
```

Manejadores de nodos

`insert` debe retornar el nodo insertado. Ese objeto funcionará como manejador para ejecutar posteriormente `decrease-key` o `delete` sin buscar linealmente la clave.

6. Invariantes que deben comprobarse

Después de cada operación de prueba, el equipo deberá verificar:

1. **Orden de montículo:**

$$x.parent \neq \text{None} \implies x.parent.key \leq x.key.$$

2. **Grado correcto:** degree coincide con la cantidad de hijos.3. **Relaciones consistentes:** el padre de cada hijo es el nodo esperado.4. **Lista circular válida:**

$$x.right.left = x, \quad x.left.right = x.$$

5. **Mínimo correcto:** H.min apunta a una raíz de clave mínima.6. **Tamaño correcto:** H.n coincide con la cantidad total de nodos alcanzables.7. **Marcas válidas:** las raíces no permanecen marcadas.8. **Consolidación:** después de `extract-min`, no existen dos raíces con el mismo grado.

7. Distribución sugerida de los 120 minutos

Etapas	Actividad	Tiempo
Inicio	Revisión del trabajo preparatorio, interfaz e invariantes	10 min
Resuelto 1	Instrumentación de potencial y costo amortizado	18 min
Resuelto 2	Consolidación programada por grados	22 min
Resuelto 3	Cola basada en <code>heapq</code> con actualización perezosa	20 min
Propuesto 1	Completar, probar y depurar el montículo de Fibonacci	30 min
Propuesto 2	Comparación experimental e interpretación	15 min
Cierre	Verificación individual y organización de evidencias	5 min

8. Ejercicios resueltos

8.1. Ejercicio resuelto 1: instrumentar la función potencial

El primer programa representa únicamente las variables necesarias para estudiar el costo amortizado. No reemplaza al montículo completo, pero permite comprobar los cálculos de las diapositivas.

Listing 3: Cálculo programado del potencial y costo amortizado

```

1 from dataclasses import dataclass
2
3
4 @dataclass(frozen=True)
5 class EstadoPotencial:
6     arboles: int
7     marcados: int
8
9     def potencial(self) -> int:
10         return self.arboles + 2 * self.marcados
11
12
13 def costo_amortizado(costo_real, antes, despues):
14     delta = despues.potencial() - antes.potencial()

```

```

15     return costo_real + delta, delta
16
17
18 antes = EstadoPotencial(arboles=4, marcados=3)
19 despues = EstadoPotencial(arboles=7, marcados=0)
20
21 amortizado, delta = costo_amortizado(4, antes, despues)
22
23 print("Phi antes:", antes.potencial())      # 10
24 print("Phi despues:", despues.potencial())  # 7
25 print("Delta Phi:", delta)                  # -3
26 print("Costo amortizado:", amortizado)      # 1
27
28 assert antes.potencial() == 10
29 assert despues.potencial() == 7
30 assert delta == -3
31 assert amortizado == 1

```

Interpretación. El costo real simplificado es 4, pero la operación consume tres unidades de potencial. Por eso:

$$\hat{c}_i = 4 + (7 - 10) = 1.$$

El resultado describe la contabilidad amortizada, no los nanosegundos consumidos por Python.

8.2. Ejercicio resuelto 2: consolidación programada por grados

El siguiente programa modela únicamente raíces. Cada raíz registra su clave, grado e hijos. Cuando dos raíces tienen el mismo grado, la de mayor clave se enlaza debajo de la menor.

Listing 4: Consolidación simplificada de raíces

```

1 from dataclasses import dataclass, field
2
3
4 @dataclass
5 class Raiz:
6     clave: int
7     grado: int = 0
8     hijos: list = field(default_factory=list)
9
10
11 def enlazar(a, b):
12     padre, hijo = (a, b) if a.clave <= b.clave else (b, a)
13     padre.hijos.append(hijo)
14     padre.grado += 1
15     return padre
16
17
18 def consolidar(raices):
19     por_grado = {}
20
21     for raiz in raices:
22         actual = raiz
23         while actual.grado in por_grado:
24             otra = por_grado.pop(actual.grado)
25             actual = enlazar(actual, otra)
26             por_grado[actual.grado] = actual
27
28     return list(por_grado.values())
29

```

```

30
31 raices = [Raiz(23), Raiz(7), Raiz(21),
32           Raiz(18, 1), Raiz(52), Raiz(38, 1),
33           Raiz(17, 1), Raiz(24, 2)]
34
35 resultado = consolidar(raices)
36 grados = sorted(r.grado for r in resultado)
37
38 print([(r.clave, r.grado) for r in resultado])
39 assert len(grados) == len(set(grados))
40 assert min(r.clave for r in resultado) == 7

```

Qué debe observarse:

- el diccionario `por_grado` cumple el papel del arreglo auxiliar de consolidación;
- un enlace puede aumentar el grado y producir una nueva colisión;
- al finalizar, no existen dos raíces con el mismo grado;
- esta versión es didáctica y no implementa todavía listas circulares ni punteros de padre.

8.3. Ejercicio resuelto 3: simular `decrease-key` con `heapq`

`heapq` no permite localizar y disminuir directamente la prioridad de un elemento arbitrario. Una solución práctica es invalidar la entrada anterior e insertar otra con la nueva clave.

Listing 5: Cola de prioridad con actualizaciones perezosas

```

1 import heapq
2 import itertools
3
4 REMOVED = object()
5
6
7 class HeapqPriorityQueue:
8     def __init__(self):
9         self.heap = []
10        self.entries = {}
11        self.counter = itertools.count()
12
13    def insert(self, item, priority):
14        if item in self.entries:
15            self.remove(item)
16        entry = [priority, next(self.counter), item]
17        self.entries[item] = entry
18        heapq.heappush(self.heap, entry)
19
20    def remove(self, item):
21        entry = self.entries.pop(item)
22        entry[2] = REMOVED
23
24    def decrease_key(self, item, new_priority):
25        old_priority = self.entries[item][0]
26        if new_priority > old_priority:
27            raise ValueError("La nueva clave debe ser menor")
28        self.insert(item, new_priority)
29
30    def minimum(self):
31        self._discard_removed()

```

```
32     priority, _, item = self.heap[0]
33     return item, priority
34
35     def extract_min(self):
36         while self.heap:
37             priority, _, item = heapq.heappop(self.heap)
38             if item is not REMOVED:
39                 del self.entries[item]
40                 return item, priority
41             raise IndexError("Cola vacia")
42
43     def _discard_removed(self):
44         while self.heap and self.heap[0][2] is REMOVED:
45             heapq.heappop(self.heap)
46
47
48 pq = HeapqPriorityQueue()
49 pq.insert("a", 20)
50 pq.insert("b", 12)
51 pq.insert("c", 30)
52 pq.decrease_key("c", 5)
53
54 assert pq.minimum() == ("c", 5)
55 assert pq.extract_min() == ("c", 5)
56 print("Activos:", len(pq.entries))
57 print("Tamano fisico:", len(pq.heap))
```

Lectura de resultados

Después de una actualización puede cumplirse:

tamaño físico de `heapq` > cantidad de elementos activos.

La diferencia corresponde a entradas obsoletas que todavía no llegaron a la raíz para ser descartadas.

9. Ejercicios propuestos

9.1. Ejercicio propuesto 1: implementar y validar el montículo de Fibonacci

Complete la clase indicada en la sección de especificación. Todas las operaciones deberán ejecutarse sobre nodos enlazados y conservar las invariantes.

Requisitos funcionales:

1. `insert` devuelve el nodo insertado como manejador;
2. `minimum` retorna el nodo mínimo sin retirarlo;
3. `extract-min` promueve hijos, elimina el mínimo y consolida;
4. `decrease-key` realiza corte simple o cortes en cascada cuando corresponda;
5. `union` concatena listas de raíces sin reinserciones;
6. `delete` reutiliza disminución de clave y extracción del mínimo;
7. un método `validate()` comprueba todas las invariantes;

8. un método `stats()` devuelve n , $t(H)$, $m(H)$ y $\Phi(H)$.

Pruebas obligatorias:

1. inserciones crecientes, decrecientes, aleatorias y con claves repetidas;
2. extracción completa y comprobación del orden no decreciente;
3. unión con montículos vacíos y no vacíos;
4. disminución sin corte, con corte simple y con al menos dos cortes en cascada;
5. eliminación de una raíz y de un nodo interno;
6. reproducción de los ejemplos de `extract-min` y `decrease-key` de las diapositivas;
7. después de cada operación, ejecución de `validate()`.

El equipo deberá instrumentar contadores de enlaces, cortes, consolidaciones y raíces examinadas, sin incluir mensajes de impresión dentro de las mediciones de tiempo.

9.2. Ejercicio propuesto 2: comparar con `heapq` y aplicar en Dijkstra

Compare la implementación del montículo de Fibonacci con la cola resuelta basada en `heapq`. Ambas deberán recibir exactamente la misma secuencia de operaciones.

Escenario A: operaciones básicas

Para $n \in \{1\,000, 5\,000, 10\,000, 25\,000\}$:

1. genere n claves con una semilla fija;
2. mida por separado inserción y extracción completa;
3. compruebe que ambas estructuras devuelven el mismo orden;
4. realice al menos 10 repeticiones y reporte mediana, mínimo y máximo.

Escenario B: muchas disminuciones de clave

Para cada n :

1. inserte n elementos identificables;
2. realice $q = 10n$ disminuciones válidas sobre elementos activos;
3. ejecute una extracción cada 20 disminuciones;
4. registre tiempo, cortes, enlaces, árboles, marcados, entradas activas y tamaño físico de `heapq`;
5. explique si aparece una ventaja práctica del montículo de Fibonacci.

Escenario C: Dijkstra

Implemente Dijkstra con ambas colas de prioridad y utilice las mismas instancias, pesos y vértice fuente. Evalúe grafos dispersos con $|E| \approx 3|V|$ y grafos de densidad intermedia con $|E| \approx 10|V|$. Compruebe que ambas versiones producen las mismas distancias y contraste:

$$\mathcal{O}((|V| + |E|) \log |V|)$$

con montículo binario, frente a:

$$\mathcal{O}(|E| + |V| \log |V|)$$

con montículo de Fibonacci.

Hipótesis experimental

No se debe asumir que el montículo de Fibonacci será más rápido. `heapq` utiliza una representación compacta sobre listas, mientras la implementación académica maneja objetos y varios punteros por nodo. La conclusión deberá distinguir costo asintótico, costo amortizado, constantes de implementación y resultados medidos.

10. Metodología de medición

Listing 6: Función básica para medir una operación

```

1 from time import perf_counter_ns
2 from statistics import median
3
4
5 def medir(funcion, repeticiones=10):
6     tiempos = []
7     for _ in range(repeticiones):
8         inicio = perf_counter_ns()
9         funcion()
10        fin = perf_counter_ns()
11        tiempos.append(fin - inicio)
12    return {
13        "min_ns": min(tiempos),
14        "mediana_ns": median(tiempos),
15        "max_ns": max(tiempos),
16    }
```

Para disminuir ruido experimental:

- no incluya generación de datos dentro del intervalo medido;
- reconstruya estructuras independientes para cada repetición;
- use la misma semilla y secuencia de operaciones;
- verifique la corrección antes de interpretar tiempos;
- reporte versión de Python, sistema operativo y características básicas del equipo;
- evite concluir a partir de una sola ejecución.

11. Resultados mínimos requeridos

Los gráficos mínimos son:

1. tiempo total frente a n para inserción y extracción;
2. tiempo total frente a q para el escenario de muchas disminuciones;
3. tamaño físico de `heapq` frente a entradas activas;
4. tiempo de Dijkstra frente a $|V|$, diferenciando densidad del grafo.

12. Preguntas de análisis

Responda con evidencia de los experimentos:

1. ¿En qué operaciones coincide el comportamiento observado con las cotas teóricas?
2. ¿Por qué `heapq` puede ser más rápido aunque `decrease-key` sea $\mathcal{O}(\log n)$?
3. ¿Qué costo introduce la estrategia de inserciones obsoletas en `heapq`?
4. ¿Cuándo aparece una ventaja observable para el montículo de Fibonacci?
5. ¿Cómo cambia $\Phi(H)$ durante inserciones, consolidaciones y cortes?
6. ¿Por qué una operación con varios cortes puede seguir teniendo costo amortizado $\mathcal{O}(1)$?
7. ¿Qué errores de implementación detectaron los verificadores de invariantes?
8. ¿Recomendaría esta implementación para un sistema real en Python? Sustente la decisión.

13. Organización del trabajo grupal

Todos los integrantes deberán poder explicar y ejecutar al menos:

- una consolidación;
- un corte simple;
- un corte en cascada;
- el cálculo de la función potencial;
- una diferencia concreta entre `heapq` y el montículo de Fibonacci.

14. Producto de entrega

Producto de entrega

El equipo entregará:

1. código fuente completo y organizado, tanto ejercicios resueltos como propuestos;
2. pruebas automatizadas de operaciones e invariantes;

3. implementación de referencia basada en **heapq**;
4. generador reproducible de secuencias de operaciones y grafos;
5. gráficos solicitados;
6. informe PDF con lo anterior y descripción breve con metodología, resultados y respuestas de análisis;
7. conclusiones grupales de 250 a 350 palabras.

15. Tabla de evaluación

Criterio	Indicador de logro	Puntaje
Asistencia y participación en el laboratorio	Asiste, permanece durante la práctica y participa en implementación, pruebas, análisis y revisiones. Se determina individualmente.	4
Corrección y funcionamiento	El montículo de Fibonacci, la referencia con heapq , las pruebas y los verificadores funcionan; las operaciones preservan invariantes y ambas estructuras producen resultados equivalentes.	8
Análisis de complejidad y resultados	Relaciona costo real, potencial y costo amortizado; interpreta los experimentos sin confundir complejidad asintótica con rendimiento observado.	3
Dominio individual	Explica la estructura, consolidación, marcas, cortes, función potencial y comparación con heapq ; ejecuta o modifica una prueba durante la revisión.	4
Total		19

16. Casos mínimos de prueba

- extraer mínimo de una estructura vacía y de un solo nodo;
- claves duplicadas;
- unión con un montículo vacío;
- disminuir una clave al mismo valor;
- rechazar una clave nueva mayor que la actual;
- disminuir la clave de una raíz;
- corte cuyos padres no estaban marcados;
- corte en cascada de por lo menos dos niveles;
- eliminar un nodo interno con hijos;
- secuencias aleatorias contrastadas con **heapq**;
- Dijkstra en un grafo desconectado;
- aristas de peso cero y rechazo de pesos negativos.

17. Referencias

Referencias

- [1] Fredman, M. L. y Tarjan, R. E. (1987). Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, 34(3).
- [2] Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). *Introduction to Algorithms* (4.^a ed.). MIT Press.
- [3] Python Software Foundation. *heapq — Heap queue algorithm*. <https://docs.python.org/3/library/heapq.html>
- [4] MIT CSAIL. *Fibonacci Heaps, 6.854 Notes*. <https://courses.csail.mit.edu/6.854/21/Notes/n01-fibonacci.html>
- [5] Princeton University. *Fibonacci Heaps*. <https://www.cs.princeton.edu/~wayne/teaching/fibonacci-heap.pdf>